

School of Computer Science and Engineering, UNSW



Never Stand Still

WK08-01: Smart Contract Testing

Software Architecture for Blockchain Applications,
COMP6452

Salil Kanhere, Nadeem Ahmed

Agenda

Importance and types of testing

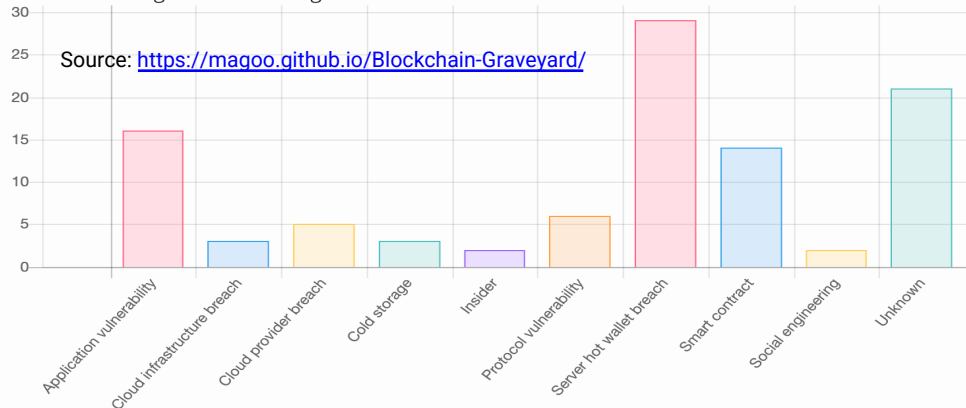
Common vulnerabilities & solutions

Tools & techniques

- We'll discuss why it is important to test smart contracts or any software for that matter.
- Then different types of testing are introduced.
- We discuss a common set of vulnerabilities and solutions before we talk about tools and techniques for smart contract testing.

Failures in Blockchains are Permanent & Catastrophic ROOT CAUSE ESTIMATES

The data below is roughly gleaned from publicly available data about **94** incidents. This should assist estimation during threat modeling.



WK 08-01

3



- Failures in blockchains are permanent and catastrophic.
- Here are some statistics from a website called Blockchain Graveyard.
- We can see the application and smart contract vulnerabilities, as well as how you manage keys are major key sources of attacks.
- More frighteningly, for 22 of cases we don't even know what took place. Those are under "unknowns".
- There are also issues around system-level vulnerabilities like protocol and cloud vulnerabilities.
- To guard against these issues:
 - First, we need to build secure software and infrastructure.
 - Then we should test them.

Scope of Blockchain-based Applications Security

Smart contracts	Cryptography & Key management
Blockchain platform	Access control & KYC
• Consensus • Smart contract language & execution-environment • Cryptography	Privacy
DApp architecture	Infrastructure, especially for private blockchains
Integration with external systems	Business continuity & disaster recovery
Data management	Governance & compliance

WK 08-01

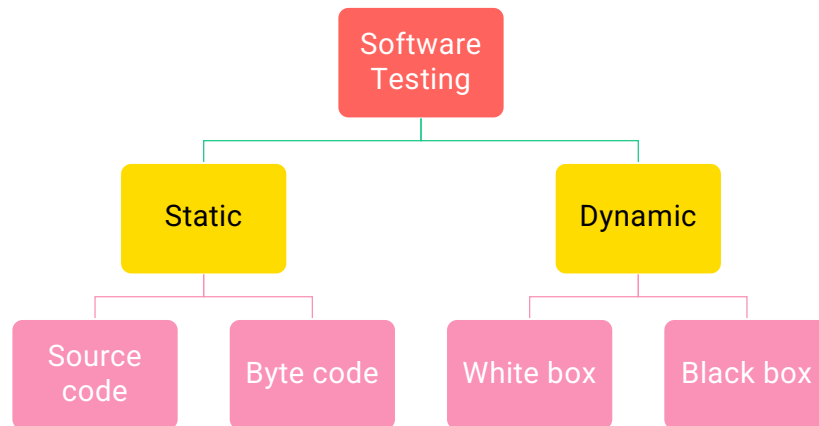
4



- To make sure a blockchain-based application is secure, we need to conduct a large set of tests. What specific tests and depth of testing can depend on the importance of the application. Typically, the final phase of testing is done by a 3rd party.
- When we think about blockchain security, most people only consider smart contract security. Security testing of smart contracts includes both code-level testing and run-time testing.
 - In this class, we'll focus mostly on smart contract testing.
- But as we saw from the previous graph, to make sure a blockchain-based application is secure, we need to make sure many aspects/components of the system are secure.
- This is ensured by **secure design and extensive testing**.
- The choice of the blockchain platform is the main consideration. The key focus should be the blockchain platform's consensus mechanism, its use of cryptography, and smart contract language and execution environment
 - If the blockchain platform is well-established and time-tested, you have some level of assurance and a good understanding of how it works.
 - If it's a new platform, then you need to review and test everything from its design, protocols, and smart contract execution environment.
- Software architecture should be designed to achieve security (i.e., secure by design). Architecture testing is a qualitative review of the design to determine its soundness in dealing with potential security issues.
 - While typical architecture reviews consider security as a non-functional requirement, there should be a separate detailed review focusing only on security.
- Integration with external systems, how we manage data, cryptography and key management, access control and KYC, and privacy are other design considerations. They can also be tested once developed.
- We should choose, configure, and test the infrastructure chosen to host the blockchain and other components of the application. This is particularly important for private blockchains.
- Business continuity and disaster recovery are also important. If you are hosting the blockchain platform you need to think about the number of nodes and where they are located to enhance availability.
- Blockchain governance is all about how you manage the platform from the addition of a new node and users, updating smart contracts to who pay for what and how issues are resolved.
- If your application needs to comply with certain regulations, then you may also have some reporting requirements to show compliance. While some authority may run a node, you may still have other obligations to ensure you design the system considering such compliance needs and continue to operate satisfying these compliance needs.

KYC – Know Your Customer

Types of Software Testing



- There are several classifications of software testing. Here's one way, that I would consider more relevant to smart contracts.
 - Another form of classification is functional and non-functional testing.
- Static testing - is a class of methods that examine the source code or bytecode of a contract without executing it. Essentially code testing.
 - Source code – use code as it is. Typically IDEs (e.g., Remix) give various hints as you write code. Or evaluated at the time of compilation.
 - Byte code – Use the compiled code, e.g., when multiple high-level languages can generate the same byte code.
- Dynamic testing – observes a contract while executing it in a test or production environment. Essentially, it is run-time testing.
 - White box testing – You know code or intentional functionality.
 - Black box testing – Only ABI is available so you know the functions and parameters.
 - Unit testing and integration testing are forms of dynamic testing as we execute the code.

Agenda

Importance & types of testing

Common vulnerabilities & solutions

Tools & techniques

Let's focus a bit on some of the common vulnerabilities and how we can overcome them

We have already discussed a couple of these issues to different extent through the course

Known Issues/Vulnerabilities in SC

Race conditions

- Reentrancy
- Cross-function race conditions
- Deadlocks

Denial of Service (DoS)

- Unexpected throw
- Gas limit
 - Block & TX limits

Arithmetic overflow/underflow

TX order dependence (front running)

Timestamp & block no dependence

- Random no

Access control

- Ability to call `selfdestruct()`

Bad error handling

Language-specific behaviour

- Not setting owner of a contract
- Depreciated functions
- Short address attack in EVM
- Call stack depth

WK 08-01

7



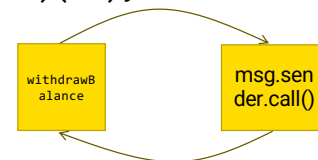
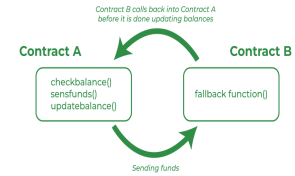
- These are many well-known issues in smart contracts. In the next couple of slides, we will discuss the ones in blue.
- A race condition occurs when more than one piece of code tries to update the same ledger state concurrently.
 - Race conditions occur when 2 or more functions try to update the state and the final result depends on which function runs last.
 - Re-entrancy attack occurs when you call another smart contract, which can call your contract again to access its state. (For example, A contract transfers Ether to another contract and hands control over to it and is called back from this contract before the transaction has been completed.)
 - Deadlocks could make a smart contract unusable though it is not destroyed, e.g., the contract gets stuck at some state.
- Denial of Service is possible when you do not properly handle errors or due to the block gas limit. We will talk about a few examples.
- Arithmetic overflow and underflow of variables is another problem where numbers stored in variables overflow and underflow.
- There can be unintended behaviour when your smart contract is sensitive to transaction orders. One such example is front running.
- There are issues about the use of time. One thing is block timestamp is not reliable. Another is if others know (by looking at your smart contract and its code), that you are planning to do something at a specific block number or hash of that block, miners could try to build a block with specific properties to make use of your contract.
- Not doing proper access control could allow others to run away with crypto or destroy your contract by calling the `selfdestruct` function.
- Due to bad error handling you may open up vulnerabilities that others may take advantage of or it could lead to a deadlock.
- There can also be smart contract language-specific issues:
 - E.g., if you forget to set the owner of a smart contract anyone can change its state.
 - Also, the use of depreciated functions is another problem, which can go unnoticed depending on the Solidity compiler version you use.
 - There were specific issues related to how EVM handle certain addresses and limits on function depth. Short Address Attack is a vulnerability where attackers exploit the characteristic of address encoding algorithms to ignore the trailing characters of the encoded string. By constructing a specific encoded string that completely matches the prefix portion of a legitimate address, attackers deceive users into using the address controlled by the attacker when sending funds or performing operations.
- Now that you know these, you should definitely try to check for them. There can also be many others that are specific to a given smart contract. Hence, you need to check for those as well.

Reentrancy

```
mapping (address => uint) private userBalances;

function withdrawBalance() public {
    uint amountToWithdraw = userBalances[msg.sender];
    (bool success, ) =
        msg.sender.call.value(amountToWithdraw)("");
    require(success);
    userBalances[msg.sender] = 0;
}
```

Source <https://scsfq.io/hackers/reentrancy/>



WK 08-01

8



- Reentrancy is a type of attack that can occur in smart contracts that allow untrusted external code to be executed within the contract. This can happen when a smart contract calls an external contract, and the external contract then calls back into the original contract, potentially causing an infinite loop. A reentrancy attack is a method of exploiting a vulnerability in a smart contract that allows an attacker to repeatedly call a function in the contract, causing an infinite loop and potentially stealing funds.
- This is an example of re-entrancy within a single function in Solidity. This is what took place in The DAO attack.
- This contract allows the *msg.sender* to withdraw its ETH balance.
- It first finds the balance and tries to transfer the ETH balance to *the msg.sender's* address using *call.value* function.
- When you use the *call* function, you may be calling another contract, which in turn may call your *withdrawBalance* function again. As you still have a balance, this will lead to another call to that contract, which may call *withdrawBalance* again, forming a loop.
- Once the loop is broken, you end up transferring the amount to withdraw multiple times.
- In this example, the external call aims to transfer ETH value to the *msg.sender*. It's crucial to note that the user's balance is updated to zero only after the external call. Therefore, if the *msg.sender* happens to be another smart contract, it can, through its fallback function, recall the *withdraw* function. Since the user balance mapping isn't updated until after the call, the reentering invocation of *withdraw* can repeatedly withdraw the designated balance, eventually depleting the smart contract of its total funds. In the final step, the attacker merely needs to check the overall ETH balance of their target smart contract to ensure they don't attempt to withdraw more ETH than the contract holds, as that would revert the entire transaction along with its state changes. Additionally, the attacker needs to consider the execution depth and its accumulated gas cost to not run out of gas during the repeated calls.
- There are at least 2 ways to fix this:
 - We need to move the line that set the user balance to zero before the *call.value*. That way, even if that called contract calls you back, the balance is zero. Hence, they can't withdraw anything more than what they had as a balance.
 - An alternative fix is to use a *withdraw* function (a new function) to get the receiver to pull the crypto than us pushing the funds.
- Reentrancy can happen across multiple functions too.

Cross Function Reentrancy

```
mapping (address => uint) private userBalances;
mapping (address => bool) private claimedBonus;
mapping (address => uint) private rewardsForA;

function withdrawReward(address recipient) public {
    uint amountToWithdraw = rewardsForA[recipient];
    rewardsForA[recipient] = 0;
    (bool success, ) = recipient.call.value(amountToWithdraw)("");
    require(success);
}

function getFirstWithdrawalBonus(address recipient) public {
    require(!claimedBonus[recipient]);
    rewardsForA[recipient] += 100;
    withdrawReward(recipient);
    claimedBonus[recipient] = true;
}
```

Source: <https://scsfg.io/hackers/reentrancy/>

- Here's another example where just setting balance before call won't work.
- The *withdrawReward* function is fixed to overcome reentrancy issue.
- However, it can be called within *getFirstWithdrawalBonus* function, where for the 1st withdrawal you get a bonus.
- While the *call.value* is pending you can call *getFirstWithdrawalBonus* function.
- In this case, before calling the withdraw function *claimedBonus* need to be set to *True*.
- Another potential solution is to use a mutex.

Front-Running

All TXs are visible while they wait in the TX (mem) pool

- Waiting time varies from seconds to hours

An adversary can issue a TX with a much higher TX fee

- Displacement – Bidding, crypto exchanges, registering a domain name
- Insertion – Seeing a buy order, buy first & sell immediately at a higher price (aka sandwich attack)
- Suppression – Slowing others down

Solutions

- Restructure SCs such that order or time is not a factor that determines outcome
- Commit & reveal, e.g., hash lock
- Zero-knowledge protocols

See <https://scsfq.io/hackers/frontrunning/>

Header
Buy 1000 XYZ Token
Buy 1000 XYZ Token
Sell 1000 XYZ Token

WK 08-01

10



- Frontrunning issue occurs because all transactions are visible to anyone while they await in the transaction pool.
- Waiting time could vary from a few seconds to many hours. In the meantime, others can see what your transaction is planning to do and adapt their transactions.
- An adversary has a good chance of getting his/her transaction included before yours if they offer to pay a (much) higher transaction fee than yours. Even if they don't offer a high transaction fee, they can do by colluding with miners who may be successful in building a block with adversarial transactions before your transaction is included in a block.
 - Miners may do this on their own too. Gains from such attacks are called *miner extracted value* (MEV).
- There are several scenarios here:
 - Displacement – where the adversary replaces your transaction with his/her transaction by outbidding you. This can be a problem in bidding, crypt exchanges (particularly distributed ones), and registering for resources like domain names.
 - Insertion - Adversary insets one or more TXs before your transaction. E.g., knowing that you are interested in buying something big, they can buy that first and immediately sell it to you at a profit. Sometimes you can even see these adversarial transactions are sandwiched between a legitimate transaction.
 - Suppression - adversary's objective is to slow you down. They can do this by submitting large and high fee-paying transactions.
- Potential solutions include:
 - Modifying your smart contract such that it doesn't depend on order or time of transactions. This is not always possible.
 - In such cases, you can use solutions like commit and reversal or even zero-knowledge proofs. The idea is to hide sensitive information until the desired result is achieved. E.g., you may commit to buying something at a set price. Based on this you send a hash value. Later revealing the actual price such that the smart contract can check with the previous hash.
 - Ethereum is planning to encrypt the transaction pool and also the transfer of transactions between 2 nodes.

Arithmetic Overflow/Underflow

```
mapping (address => uint256) public balanceOf;

function transfer(address _to, uint256 _value) {
    require(balanceOf[msg.sender] >= _value);
    balanceOf[msg.sender] -= _value;
    balanceOf[_to] += _value;
}

function transfer(address _to, uint256 _value) {
    require(balanceOf[msg.sender] >= _value &&
        balanceOf[_to] + _value >= balanceOf[_to]);
    balanceOf[msg.sender] -= _value;
    balanceOf[_to] += _value;
}
```

Another solution is to use
SafeMath.sol library

Source: <https://github.com/Consensys/smart-contract-best-practices/blob/master/docs/attacks/insecure-arithmetic.md>

- This is the usual case of over or underflowing a variable.
- Also, be aware this can happen with ++, --, *, /, and bit shift operations
- Be careful with the smaller data types like uint8, uint16, uint24...etc: they can even more easily hit their maximum value.
- The solution is to check if the sender has a balance and for overflows at the receiver.
- Another solution is to rely on a library like SafeMath that performs these checks for you.
- If a balance reaches the maximum uint value (2^{256}) it will circle back to zero which checks for the condition. This may or may not be relevant, depending on the implementation. Think about whether or not the uint value has an opportunity to approach such a large number. Think about how the uint variable changes state, and who has authority to make such changes. If any user can call functions which update the uint value, it's more vulnerable to attack. If only an admin has access to change the variable's state, you might be safe. If a user can increment by only 1 at a time, you are probably also safe because there is no feasible way to reach this limit.
- The same is true for underflow. If a uint is made to be less than zero, it will cause an underflow and get set to its maximum value.

Timestamp Dependence

Miners can manipulate block timestamps to some extent

Inter-block time can substantially vary

Avoid using `block.timestamp` or `block.number` x inter-block time

- Even `block.number` isn't future proof

Solutions

- Design SC to not rely on time
- Use oracle as a time source

See: <https://github.com/Consensys/smart-contract-best-practices/blob/master/docs/development-recommendations/solidity-specific/timestamp-dependence.md>

- It is unsafe to decide the execution of your smart contract based on time as miners can manipulate block timestamps to some extent.
- If the reward of manipulating time is high, miners may even put additional computing power to set the timestamp to a value favourable to the miner.
- Also, estimating time based on inter-block time is risky.
- Hence, it's not a good idea to use either `block.timestamp` or even `block.number` x inter-block time (e.g., in Ethereum `block.number` x 12 sec)
 - Even `block.number` is not very future-safe to guard against significant changes in mining difficulty or inter-block time. For e.g., say Ethereum suddenly decide to reduce inter-block time to 5 sec, as they have already gone down from 15 sec to 12 sec.
- The best solution is not to rely on time.
- Another option is to rely on an external time source via an oracle. This way we can rely on physical time. Of course, we are trusting the oracle.

Denial of Service (DoS)

```
contract Auction {
    address currentLeader;
    uint highestBid;

    function bid() payable {
        require(msg.value > highestBid);
        require(currentLeader.send(highestBid));
        currentLeader = msg.sender;
        highestBid = msg.value;
    }
}
```

Source: <https://github.com/Consensys/smart-contract-best-practices/blob/master/docs/attacks/denial-of-service.md>

- Your smart contract can get stuck due to a bug in code or some odd sequence of function calls that you didn't anticipate.
- Here's a bidding smart contract that refunds the previous highest bidder's Ether as soon as a better bid arrives.
- Refund the old leader, if it fails then revert
 - The *fallback* function on the old leader can be set to revert any payments. If so, 2nd line will fail, reverting the transaction.
- A malicious bidder can become the leader while making sure that any refunds to their address will always fail → That bidder will always be the highest bidder.
- This is a denial of service to other potential bidders.
- If attacker bids using a smart contract which has a fallback function that reverts any payment, the attacker can win any auction. When it tries to refund the old leader, it reverts if the refund fails. This means that a malicious bidder can become the leader while making sure that any refunds to their address will always fail. In this way, they can prevent anyone else from calling the bid() function, and stay the leader forever. A recommendation is to set up a pull payment system instead,

Denial of Service (DoS) – Cont.

```
address[] private refundAddresses;  
mapping (address => uint) public refunds;  
  
function refundAll() public {  
    for(uint x; x < refundAddresses.length; x++) {  
        require(refundAddresses[x].send(refunds[refundAddresses[x]]))  
    }  
}
```

Solutions

- Pull refund than push
- Limit no of accounts
- Break the loop into multiple TXs

- Here's another example that can go wrong in 2 ways.
- Because of the *require* statement, if one participant rejects/fails, the rest of the participants can't get a payout.
- Also, if the payee list is too large, gas used by *refundAll* function may exceed the transaction gas limit.
 - Attackers could also deposit small amounts from a large number of accounts.
- Potential solutions include the use of withdraw function with a pull.
- Also, we can prevent the use of a large number of accounts by setting a limit of minimum amount or even limiting the no of accounts.
- We can also break the loop into multiple transactions, e.g., rather than refunding to 100 accounts, we can get someone to issue 10 transactions such that each transaction refund to 10 accounts.
- Another example is when a contract may iterate through an array to pay users (e.g., supporters in a crowdfunding contract). It's common to want to make sure that each payment succeeds. If not, one should revert. The issue is that if one call fails, you are reverting the whole payout system, meaning the loop will never complete. No one gets paid because one address is forcing an error.

Random Number Generation

Typical `rand()` won't generate same random no across all blockchain nodes

Use of `block.timestamp`, `block.number`, `block.difficulty`, `block.hash` as random seed is too predictable or could be manipulated

Solutions

- Use an oracle to provide random value
- Using an external commitment scheme with a smart contract, e.g. RANDAO
 - <https://github.com/randao/randao>

- Random numbers are useful in many contracts, e.g., lottery and gambling contracts.
- We can't use typical the `rand()` function available in most languages as different nodes will generate different random values → transaction fail to validate.
- While values like `block.timestamp`, `block.number`, `block.difficulty`, `block.hash` could be used to set the seed for random no generation, they are predictable for an adversary or could be manipulated by miners.
 - Even `block.hash` to some extent depend on the block difficulty. So if the stakes are high miners can look to benefit from this.
- One potential solution is to rely on an oracle.
- Another is to use some external commit scheme. For e.g., The DAO developed a scheme called RANDAO, see <https://github.com/randao/randao>

Best Practices

Avoid external calls
Finish all internal work before making external calls
Favour *pull* over *push* – Let users withdraw funds
Use *send()* over *call.value()* – *send()* has a fixed gas limit of 2,300
Exception handling – Be aware of different function behaviour
Use libraries/languages that prevent overflow & underflow
Reuse well-tested code
Avoid multi-party contracts – One party may disappear
Explicitly set visibility of functions & variables
Keep *fallback* function simple
Upgradable contracts – No hardcoding of contract addresses
Rate limiting – No of calls & crypto flows for set of blocks

- Here are some of the best practices, some of which we have already seen as patterns.
- E.g., upgradable contracts can be developed through a proxy or smart contract registry.
- There are rate-limiting patterns like speed bumps, rate limits, and balance limits as various from of limiting transactions.
- For some use cases, it may be impossible to avoid some of these, e.g., multi-party contracts or external calls. If so, understand what you are doing, and what can go wrong, and build things to around those issues.

Question

Mark True or False for each the following statements

	True	False
Blackbox testing use source code.		✓
Only way to guarantee no reentrancy is to use a mutex.		✓
Front running (i.e., TX order dependence) can be avoided by using an oracle to provide the timestamp.		✓
One way to overcome DoS while performing multiple refunds is to get recipients to withdraw funds. This is same as breaking the refund loop to a size of one.	✓	

Agenda

Importance & types of testing

Common vulnerabilities & solutions

Tools & techniques

- Next, we talk about a selected set of tools and techniques for smart contract testing.

Smart Contracts Security Testing Solutions

Tool	Blockchain				EVM		Solidity											
	TOD	Random number	Timestamp	Unpredictable state	Callstack depth	Lost Ether	Reentrancy	Unchecked call	tx.origin	Blockhash	send	selfdestruct	Visibility	Unchecked math	Costly pattern	Bad coding pattern	Deprecated	Other
MAIAN	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Manticore	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Mythril	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Osiris	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Oyente	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Porosity	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Remix-IDE	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Securify*	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
SmartCheck*	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Solgraph	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Vandal	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Source: Di Angelo, M., & Salzer, G. (2019, April). A survey of tools for analyzing Ethereum smart contracts. In 2019 IEEE Int. Conf. on Decentralized Applications and Infrastructures (DAPPCON).

WK 08-01

19



- Here's a table from a survey of testing tools for Ethereum smart contracts.
- Each row is a tool. Columns are grouped based on their purpose of testing (or objective) whether the test is performed based on bytecode or source code.
- It also split the testing based on the target, e.g., whether it's testing the blockchain platform, EVM, or the source code. Source code testing may happen at the bytecode.
- You can see that Remix-IDE has good coverage of tests.
 - However, this doesn't mean that you are covered against all possible vulnerabilities or even more complex cases of these vulnerabilities. Hence, more detailed and widespread testing is needed.
- Note that most tests are based on static testing and dynamic testing tools are less common.
- Some tools will translate or convert either byte or source code to another intermediate language that is easier to analyse using formal techniques.
- I would encourage you to read the reference paper.
- TOD (Transaction-ordering dependence, also called front running, race condition): the ordering of transactions cannot be relied on in a contract.
- Random number (also called 'nothing is secret'): has to be handled with care on a blockchain.
- Timestamp dependence: as the timestamp can be manipulated, its usage may be unsafe.
- Unpredictable state: calling a library may effect in an unforeseen change of the state of the calling contract.
- Callstack depth (until Oct 18, 2016): an external call can fail because it exceeds the maximum call depth.
- Lost Ether: Ether transferred to an orphaned address cannot be retrieved.
- Reentrancy: A contract transfers Ether to another contract and hands control over to it, and is called back from this contract before the transaction has been completed.
- Unchecked Call (incl. gasless send, bad exception handling): the low level functions call, callcode, delegatecall and send are potentially risky and should be avoided. They do not revert on failure, but return false and continue.
- tx.origin: its use for authorization is highly discouraged.
- Blockhash: can be manipulated to some degree.
- Send: can fail.
- Selfdestruct: should be checked for proper authorization.
- Visibility: the unintended visibility of functions and state variables may cause a vulnerability (e.g. leaking secrets, function exposure).
- Unchecked math (incl. integer overflow): may incur a vulnerability.
- Costly patterns (gas): can cause a transaction to fail.
- Bad coding pattern: may incur a vulnerability.
- Deprecated: disapproved Solidity elements should no longer be used.
- Other: the tool checks other issues as well.

Code Smells ^[1] – 587 contracts, 20 Code Smells

Code Smell	Definition	Code Smell	Definition
<i>Unchecked External Calls</i>	Do not check the return value of external call functions.	<i>DoS Under External Influence</i>	Throwing exceptions inside a loop which can be influenced by external users
<i>Strict Balance Equality</i>	Using strict balance equality to determine the execute logic.	<i>Unmatched Type Assignment</i>	Assigning unmatched type to a value, which can lead to integer overflow
<i>Transaction State Dependency</i>	Using tx.origin to check the permission.	<i>Re-entrancy</i>	The re-entrancy bugs.
<i>Hard Code Address</i>	Using hard code address inside smart contracts.	<i>Block Info Dependency</i>	Using block information related APIs to determine the execute logic.
<i>Nested Call</i>	Executing CALL instruction inside an unlimited-length loop.	<i>Deprecated APIs</i>	Using discarded or unrecommended APIs or instructions.
<i>Unspecified Compiler Version</i>	Do not fix the smart contract to a specific version.	<i>Misleading Data Location</i>	Do not clarify the reference types of local variables of <i>struct</i> , <i>array</i> or <i>mapping</i> .
<i>Unused Statement</i>	Creating values which never be used.	<i>Unmatched ERC-20 standard</i>	Do not follow the ERC-20 standard for ICO contracts.
<i>Missing Return Statement</i>	A function denote the type of return values but do not return anything.	<i>Missing Interrupter</i>	Missing backdoor mechanism in order to handle emergencies.
<i>Missing Reminder</i>	Missing events to notify caller whether some functions are successfully executed.	<i>Greedy Contract</i>	A contract can receive Ethers but can not withdraw Ethers.
<i>High Gas Consumption Function Type</i>	Using inappropriate function type which can increase gas consumption.	<i>High Gas Consumption Data Type</i>	Using inappropriate data type which can increase gas consumption.

[1] Chen, Jiachi, Xin Xia, David Lo, John Grundy, Daniel Xiapu Luo, and Ting Chen. "Domain Specific Code Smells in Smart Contracts," arXiv preprint arXiv:1905.01467 (2019).

- Code smells are symptoms in source code that possibly indicate deeper problems.
 - Authors manually analysed smart-contract-related posts from Stack Exchange and identified these 20 kinds of code smells.
 - Those can be categorised as related to **security, architecture, and usability problems**.
 - Authors then manually identified defined code smells in 587 contract accounts to validate the code smells.
- By checking for code smells, we can try to avoid potential bugs & improve the design of our code.
- E.g., 1st one checks whether we are validating the return value for an external call. We have already seen this can become a problem when you try to send Ether to some account using a contract.
- Other e.g., include the use of hard-coded addresses, *address.call* used in loops, high gas-consuming functions, and reentrancy.
- You should make sure your contract doesn't have these issues.

ContractFuzzer^[2] – Fuzzing SCs for Vulnerability Detection

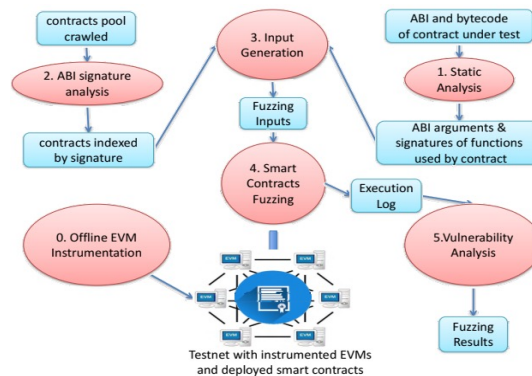
Fuzz testing – Automated testing by providing invalid, unexpected, or random data as inputs

Set of test oracles

- Gasless address.send
- Exception disorder
- Reentrancy
- Timestamp dependency
- Block no dependency
- Dangerous delegate calls
- Freezing Ether

Critique

- + Dynamic testing
- - High rate of false negatives
- - Depends on quality of test oracles



[2] Jiang, Bo, Ye Liu, and W. K. Chan. "Contractfuzzer: Fuzzing smart contracts for vulnerability detection," In Proc. 33rd ACM/IEEE Intl. Conf. on Automated Software Engineering, pp. 259-269. ACM, 2018.

- Fuzzing or fuzz testing is an automated testing technique that gives invalid or random inputs to a program and then monitors for exceptions such as crashes, failed assertions, or other potential issues.
- Groups of such inputs are called test oracles.
- ContractFuzzer generates fuzzing inputs based on the ABI specifications of a smart contract to detect security vulnerabilities.
- E.g., in a gasless send address.send() is called with value = 0.
- In exception disorder, we check whether an exception is propagated through a chain of calls.
- Freezing Ether check for cases like calling *selfdestruct* without returning Ether.
- It also uses EVM to log smart contracts' runtime behaviour and analyses these logs to identify security vulnerabilities.
- Authors tested the tool using real smart contracts and found that it works better than Oyente another popular and one of the oldest tools for smart contract testing.
- In terms of good and bad the technique:
 - This is a dynamic testing technique as the code is executed with fuzz inputs.
 - Some of the negative aspects include high false negatives, which means you can't detect a vulnerability when it exists.
 - Also, effectiveness depends on the quality of test oracles.
 - You also need a modified EVM to capture test data.

ZEUS – Analysing Safety of Smart Contracts^[3]

Automatic formal verification of SCs using abstract interpretation & symbolic model checking

Focuses on

- Correctness – Safe programming practices
- Fairness – Follow business logic

$P ::= C^*$

Technique

- Policy builder
 - Abstract language to model smart contract behaviour
- Source code translator
 - Solidity to LLVM bytecode
 - Add policy conditions as assert statements
- Verifier
 - Check assertion violations

$C ::= \text{contract } @Id \{ \text{global } v : T; \text{function}@Id(l : T) \{S\}}^*$

$S ::= (l : T@Id)^* \mid l := e \mid S;S$
 $\mid \text{if } e \text{ then } S \text{ else } S$
 $\mid \text{goto } l$
 $\mid \text{havoc } l : T \mid \text{assert } e \mid \text{assume } e$
 $\mid x := \text{post function}@Id(l : T)$
 $\mid \text{return } e \mid \text{throw} \mid \text{selfdestruct}$

[3] Kalra, Sukrit et al., "ZEUS: Analyzing Safety of Smart Contracts." In Network and Distributed Systems Security (NDSS) Symposium, Feb. 2018.

- ZEUS is a tool by IBM that focuses on automatic formal verification.
- ZEUS compare source code against its abstract representation.
- It focuses on safe programming practices (correctness) and the correctness of business logic implementation (fairness).
- First, you have to define the logic using a policy builder that provides an abstract language to model smart contract behaviour.
- Second, ZEUS translates the source code to an intermediate language called LLVM (low level virtual machine) – language independent intermediate representation (IR)
 - LLVM is a popular intermediate representation of a source code that allows the bytecode to be analysed with existing tools.
- Finally, it compares policy conditions as violations of assert conditions in the LLVM code.

ZEUS – Cont.

```
function transfer() {
  msg.sender.send(msg.value);
  balance = balance - msg.value;
}
```

Solidity code

```
<Subject> msg.sender </Subject>
<Object> msg.value </Object>
<Operation trigger="pre"> send </Operation>
<Condition> msg.value <= balance </Condition>
<Result> True </Result>
```

Policy

```
havoc balance
B@0() {
  assert(value <= balance)
  post B@0()
  balance = balance - value
}
```

Abstract Program

```
define void @transfer() {
entry:
  %__value = getelementptr %msgRecord, @msg, i32 0, i32 4
  %0 = load i256* %__value
  %1 = load i256* @balance
  %2 = icmp ule i256 %0, %1
  br i1 %2, label %"75", label %"74"

"74":
  call void @__VERIFIER_error()
  br label %"75"
; preds = %"64"
```

LLVM bitcode

Performance

- Tested with 1,524 unique contracts
- Found 94.6% of them to be vulnerable
- Test within seconds
- No false negatives & few false positives

Critique

- + Static testing
- - User needs to provide policy document
- - Can check only for pre-coded issues
- - CPU & memory expensive
- - No multi-function/contract testing

WK 00-01

23

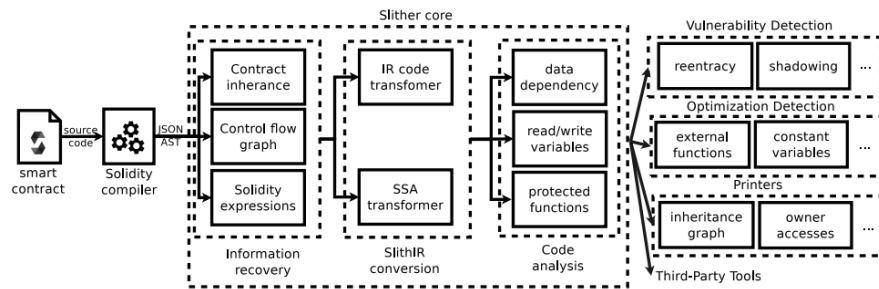


- Here's an example of a smart contract function, policy defining how it should behave, its abstract specification, and compiled LLVM bytecode.
- When tested with over 1,500 real-world contract authors found most smart contracts have known issues.
- This is a static testing technique as it uses the source code.
- The main issue is that a user needs to provide a policy document, based on which we can only check for pre-coded issues.
- While the tools runs in seconds, it need multi-core and high memory system.

Slither – A Static Analysis Framework for SCs^[4]

Use an intermediate representation called Slither

Supports security testing, code optimization, review, & user understanding



[4] Feist, Josselin, Gustavo Grieco, and Alex Groce. "Slither: a static analysis framework for smart contracts." In 2019 IEEE/ACM 2nd Intl. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8-15. IEEE, 2019.

- Slither is another static analysis tool.
- Given a compiled smart contract, it transforms the bytecode and then performs various analyses on the transformed code.
- Based on this analysis, Slither can support security testing, code optimization, review, and user understanding.
- For e.g., it can check for reentrancy, code optimisations, and provide various visualisations to understand code.

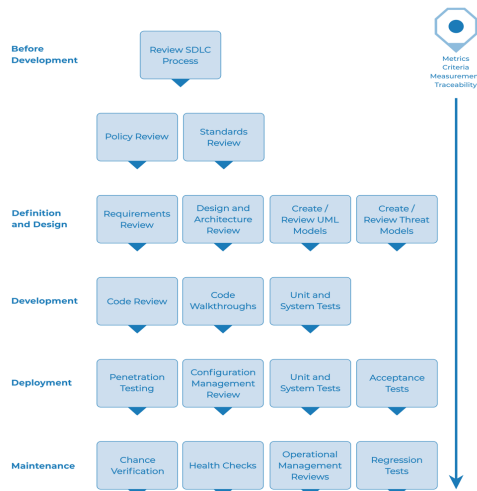
Other Tools^[5]

Tool	Level of rigor	Analysis basis	Interface Means	Description
Oyente	Heuristic	source code (.sol)	Command line-based tool	Oyente [25] is an automated security analysis tool for revealing security vulnerabilities in smart contracts.
Mythril	Analytic and Heuristic	Byte code and source code (.sol)	Command line-based tool	Mythril [29] is an automated security analysis tool for Ethereum smart contracts. It uses Concolic analysis, taint analysis and control flow checking to detect a variety of security vulnerabilities. The analysis is based on laser-Ethereum, a symbolic execution library for EVM bytecode.
Securify	Formal	Byte code and source code (.sol)	UI-based tool	Securify [22] is an automated formal security analysis tool for Ethereum smart contracts. It can detect various security issues such as input validation, reentrancy, and others.
SmartCheck	Analytic and Heuristic	source code (.sol)	UI-based tool	SmartCheck [38] is an automated static code analyzer developed by SmartDec Security Team. It runs analysis in Solidity source code and automatically checks smart contracts for security vulnerabilities and bad practices.

[5] Parizi, Reza M. et al., "Empirical vulnerability analysis of automated smart contracts security testing on blockchains." In Proc. 28th Annual Intl. Conf. on Computer Science and Software Engineering, pp. 103-113. IBM Corp., 2018.

- There are several other tools and Oyente and Mythril are popular
- All these are static analysis tools.

OWASP Testing Framework^[6]



Testing throughout SDLC

Manual inspection, threat modelling, code review, penetration testing

Identification of functional & non-functional security requirements

Details are more focused on penetration testing

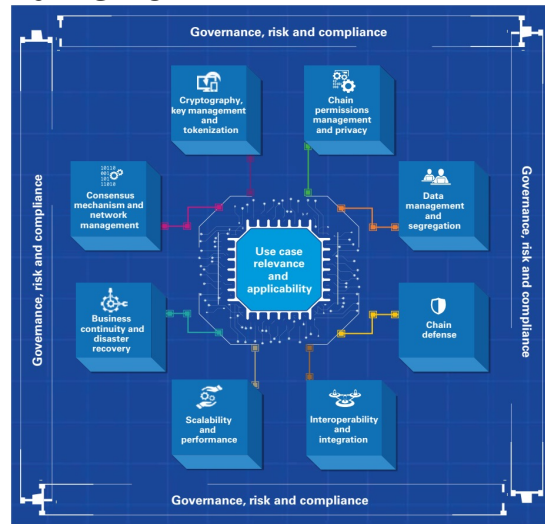
- Extensive set of test cases
- Web-application specific
- ISO/IEC 27034 cover some of the other aspects

[6] OWASP Web Security Testing Guide (WSTG) v4.2, 2020

- OWASP stands for Open Web Application Security Project.
- They provide a testing framework, as well as publish the top 10 vulnerabilities in web applications, mobile apps, and IoT.
- OWASP is perhaps the most well-known and used testing framework.
- The testing framework applies throughout the SDLC (Software Development Life Cycle) and even how you conduct SDLC.
- These are set of actions or tests that you can perform along the SDLC.
- There's some discussion on blockchain-specific OWASP, so maybe we'll see something in the future.

KPMG BC Risk Assessment Framework^[7]

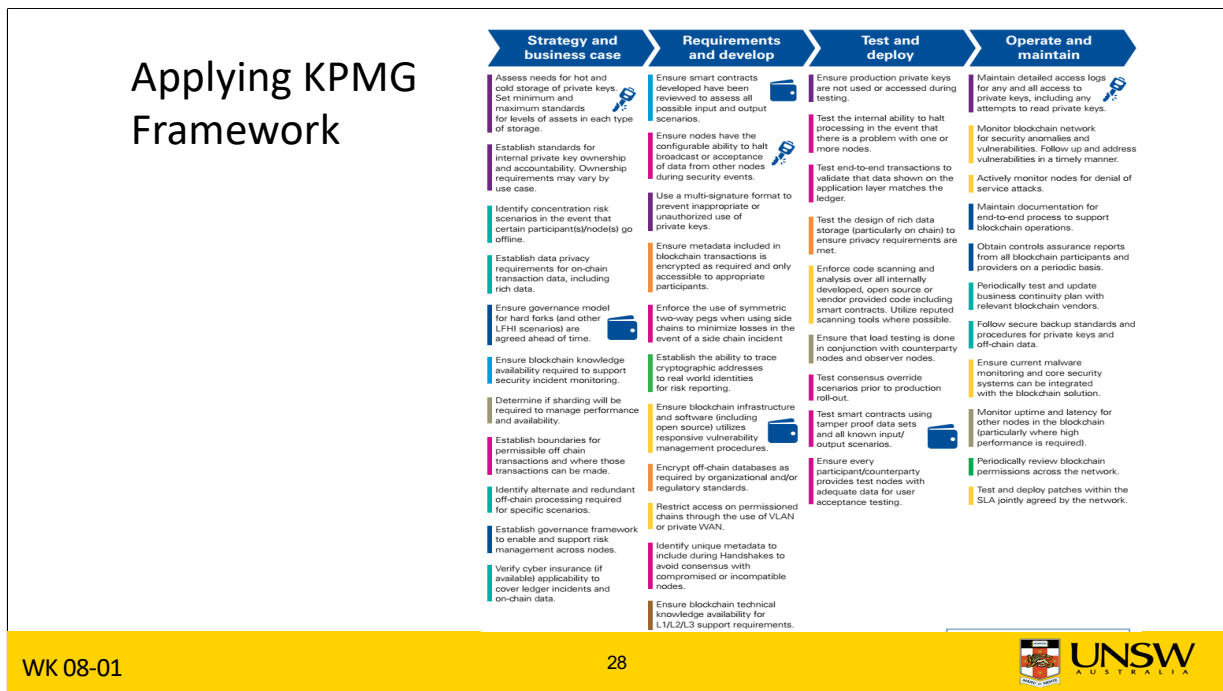
- Consensus & Network management
 - Security & incentives
- Cryptography & Key management
 - Generation, distribution, use, & revocation
- Chain Access control & privacy
- Data management & Segregation
 - On-chain vs. off-chain
- Chain defence
- Interoperability and Integration
- Scalability & Performance
- Business Continuity & DR
 - Certificate Authority (CA) & wallets
- Governance & Compliance



[7] KPMG International, Realizing Blockchains Potential, 2018

- The tools we discussed so far focus only on smart contracts. While it is a key part of the puzzle, we also need to focus on the entire application.
- KPMG blockchain risk assessment framework focuses on an entire blockchain-based application in an enterprise setting.
- This framework focuses on 8 areas from designing to testing.
- Governance and compliance is considered throughout, e.g., especially needed in regulated industries.
- All areas are analysed based on their relevance to the use case.

Applying KPMG Framework



WK 08-01

28



- Here we see a bunch of things we need to consider from business case development to development, testing, and operation of a blockchain-based application.
- These guidelines are designed for an enterprise setting. You may notice that most of these steps are common to any enterprise application.
- There are some specific steps related to blockchains such as consensus, 2-way peg, and blockchain expertise.

Question

Which of the the following statement(s) is True?

	True	False
Code Smells is a form of automated testing that use invalid, unexpected, or random data as inputs.		✓
Both static & dynamic testing are needed	✓	
We can certify a smart contract as secure if at least 2 testing tools didn't find any vulnerabilities		✓
All tools contain false positives; hence, you need to manually verify whether a vulnerability really exist	✓	

- Code smells are for programmers to avoid common pitfalls. Providing invalid, unexpected, or random data as inputs is fuzz testing
- Both static testing and dynamic testing are needed to enhance the security of an application.
- There can be false negatives. Also, no single tool can check for all possible vulnerabilities. More tools are better, but we can't say 100% secure.
- Any tool will contain false positives or positives. So, we need to check whether the problem really exists.